

COMP90086 Assignment 3 Report

Yu Luo (1528667)

Part 1 – Analysis of the Default Semi-Supervised System

The baseline system implements a **semi-supervised learning pipeline** combining an **autoencoder** for unsupervised feature learning with a **Multi-Layer Perceptron (MLP)** classifier trained on only 60 labelled samples. This design leverages the large pool of unlabelled data to learn meaningful representations that improve classification performance when labelled examples are extremely scarce.

Architecture Overview

- **Encoder:** Each MNIST image ($28 \times 28 \times 1$) is flattened into a 784-dimensional vector, then passed through a 128-unit dense layer with ReLU activation before being compressed into a 16-dimensional latent representation. This forces the encoder to learn compact features that capture the essential structural properties of handwritten digits.
- **Decoder:** The decoder mirrors the encoder architecture, reconstructing the original image from the latent vector through a 128-neuron hidden layer. Trained on 48,000 unlabelled images using **Mean Squared Error (MSE)** loss, it encourages the latent space to preserve information necessary for accurate reconstruction.
- **Classifier:** A simple MLP with one 16-neuron hidden layer (ReLU activation) and a 10-neuron output layer (softmax activation) operates on the learned latent vectors rather than raw pixels. This allows it to benefit from the feature representations learned during unsupervised pre-training, despite having access to only 60 labelled examples.

Learning Process

Training occurs in two sequential stages. First, the autoencoder learns to reconstruct digits without any label information, discovering patterns such as loops, strokes, and aspect ratios that naturally separate different digit classes in the latent space. Second, the classifier learns decision boundaries in this pre-structured latent space. Because similar digits are already clustered together by the encoder, the classifier can achieve reasonable performance with far fewer labelled examples than would be required in raw pixel space.

Quantitative Results and Observations

- **Autoencoder Final Validation MSE:** 0.014
- **Autoencoder-based Classifier Final Validation Accuracy:** 71.46%
- **Baseline Classifier Final Validation Accuracy:** 63.10%

The autoencoder demonstrates stable convergence with a final **validation MSE of 0.014**. The close alignment between training and validation loss curves (Figure 1) indicates good generalisation during unsupervised pre-training. Visual inspection of reconstructions confirms

that the network successfully captures digit topology, though some blurriness appears due to the compression bottleneck (Figure 2).

The semi-supervised approach yields substantial performance gains. The **autoencoder-based classifier achieves 71.46% validation accuracy**, significantly outperforming the **baseline classifier's 63.10%**. Both models reach 100% training accuracy, revealing overfitting on the tiny 60-sample training set. However, their generalisation differs markedly: the baseline model's validation loss climbs to 1.5180, while the autoencoder-based classifier maintains a much lower and more stable validation loss of 0.9959 (Sees Figure 4).

This 8.36 percentage point improvement strongly validates the semi-supervised learning approach. By learning from unlabelled data, the encoder creates a feature space that is more discriminative and better structured than raw pixels, enabling the classifier to generalise more effectively under extreme label scarcity.

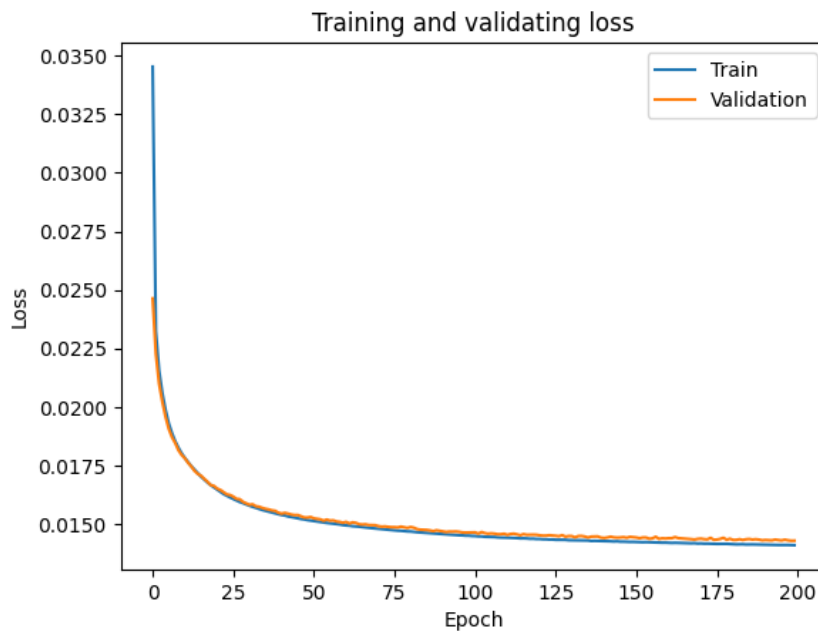


Figure 1: Autoencoder training and validation loss curves showing stable convergence.

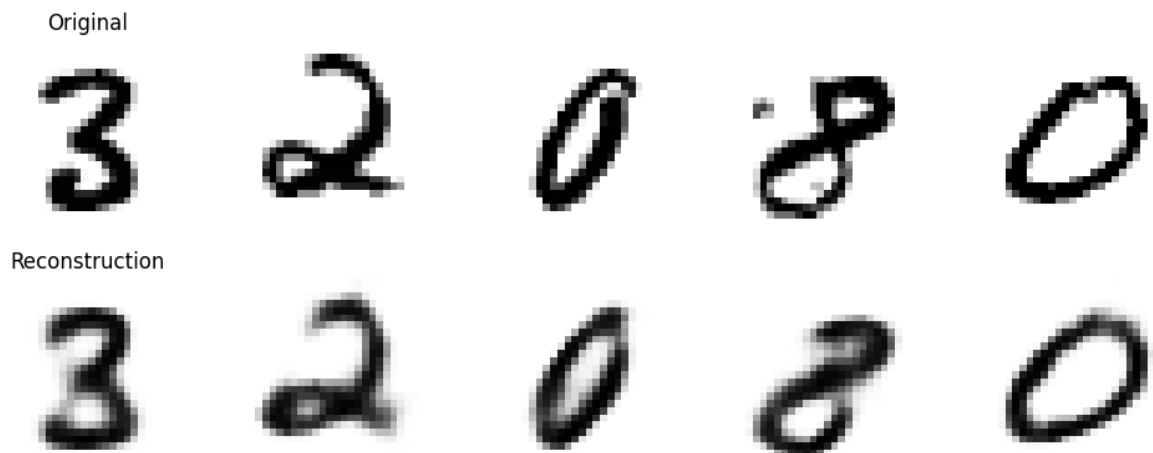
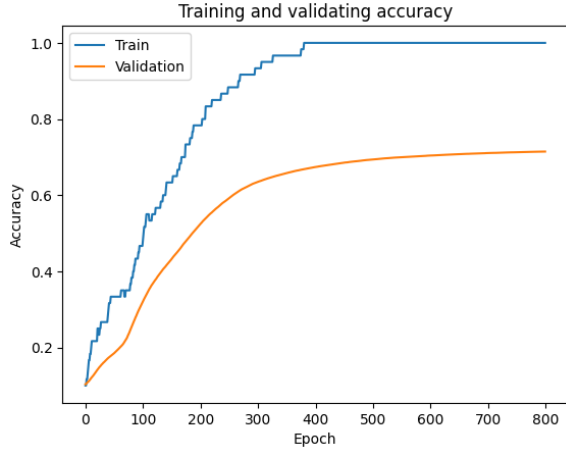
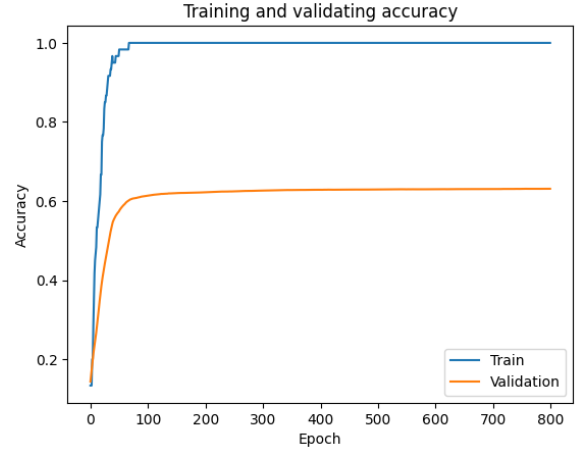


Figure 2: Original versus reconstructed images for five random test samples.

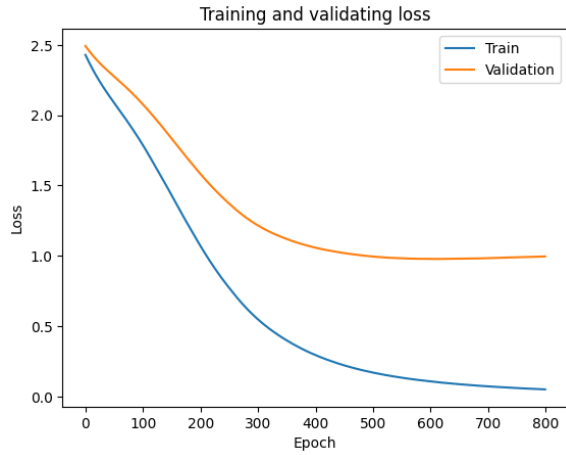


(a) Autoencoder-based Classifier

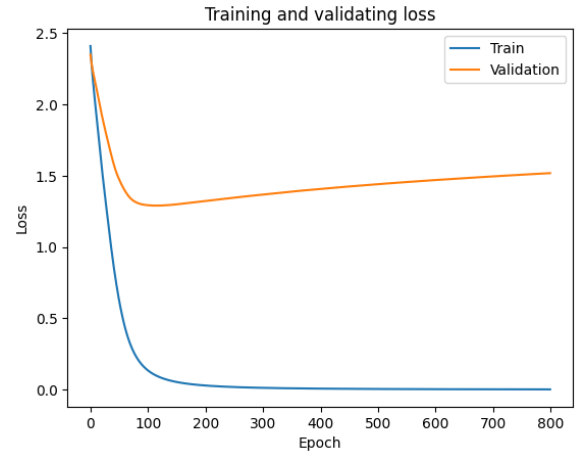


(b) Baseline Classifier

Figure 3: Validation accuracy comparison. The autoencoder-based model achieves higher accuracy.



(a) Autoencoder-based Classifier



(b) Baseline Classifier

Figure 4: Validation loss comparison showing more severe overfitting in the baseline model.

Part 2 – Search for the Optimal Latent Representation Size

Search Objective and Strategy

The goal was to identify the latent dimension that maximises classification accuracy rather than reconstruction quality. I adopted a **coarse-to-fine search strategy** to efficiently explore the dimensionality space:

1. **Coarse phase:** Tested latent sizes $\{4, 8, 16, 32, 64, 128\}$ to understand the overall performance trend across a wide range from highly compressed to over-parameterised representations.
2. **Fine phase:** After observing peak performance in the low-dimensional range, I conducted a local search around the optimal region with sizes $\{6, 10, 12, 14\}$ to pinpoint the best configuration.

All experiments used consistent training parameters (200 epochs, batch size 100) and maintained the default autoencoder and classifier architectures from Part 1 to ensure fair comparison.

Quantitative Results

Table 1: Latent dimension sweep results.

Latent Size	AE Val. Loss (MSE)	Classifier Val. Acc. (%)
2	0.0497	55.3
4	0.0308	71.9
6	0.0249	74.7
8	0.0216	73.8
10	0.0192	77.7
12	0.0168	69.5
14	0.0156	70.9
16	0.0144	69.3
32	0.0101	62.7
64	0.0079	56.7
128	0.0065	57.4

Analysis of Trends

The experiment reveals a clear tension between reconstruction fidelity and classification performance. As shown in Figure 5, reconstruction loss decreases monotonically with larger latent dimensions, as greater capacity enables the model to preserve more pixel-level detail. However, classification accuracy exhibits an **inverted-U relationship**, peaking at 10 dimensions before declining sharply.

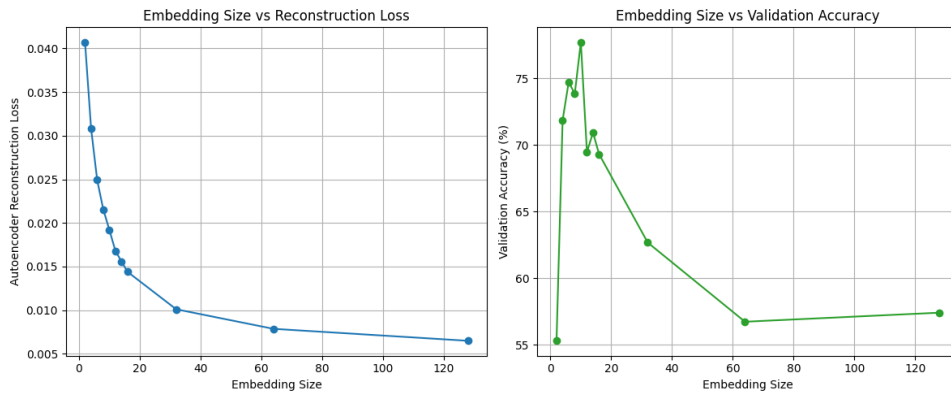


Figure 5: Classifier accuracy and reconstruction loss versus latent dimension.

This behaviour reflects three distinct regimes:

- **Under-capacity (Size ≤ 6):** The severe information bottleneck forces the encoder to discard critical structural details. Reconstructions become ambiguous (e.g., ‘4’ reconstructed as ‘9’ in Figure 6), and the loss of discriminative features directly impairs classification.

- **Optimal regularisation (Size 8–14):** The latent space is sufficiently constrained to encourage learning of abstract, generalisable features (loops, strokes, orientations) while filtering out pixel-level noise. This balance between expressiveness and compression produces embeddings highly effective for classification, peaking at 10 dimensions.
- **Over-capacity (Size ≥ 32):** The autoencoder has enough capacity to nearly memorise inputs, achieving low reconstruction error by encoding training-specific patterns and noise. This high-dimensional, noisy embedding causes the classifier to overfit, learning non-generalisable features that harm validation accuracy despite sharp reconstructions (Figure 6).

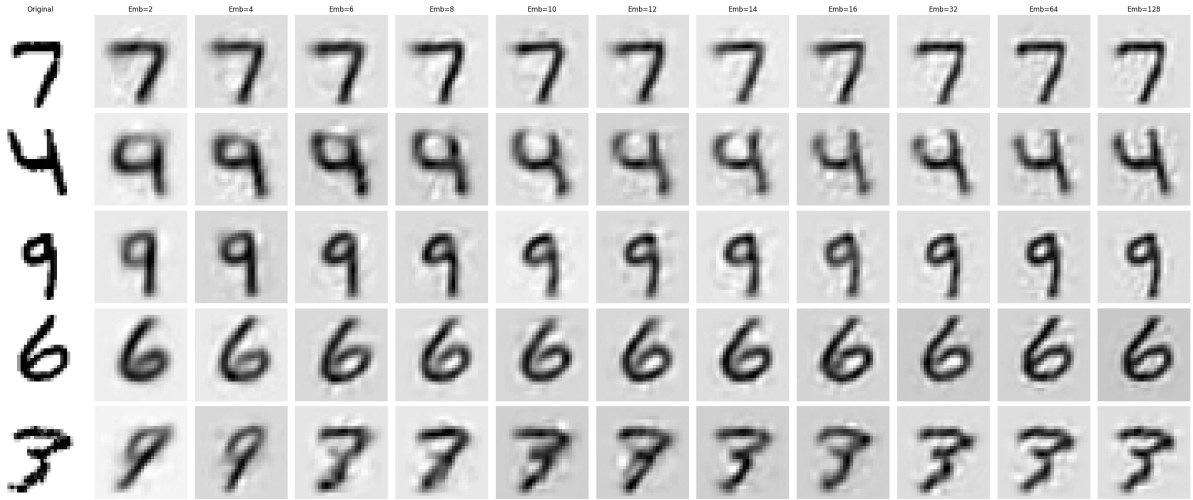


Figure 6: Reconstruction quality across different latent dimensions.

Conclusion

The optimal latent size is **10 dimensions (77.7% accuracy)**, balancing compactness and expressiveness. Crucially, reconstruction fidelity poorly predicts downstream performance—the lowest reconstruction error (128 dimensions) yielded worst classification because pixel-perfect reconstructions preserve noise and irrelevant variance that harm generalisation.

Moreover, optimal dimensionality depends on classifier capacity. With a simple 16-unit classifier, smaller latent spaces (8-14 dimensions) outperform because they provide pre-filtered, abstract features the limited model can readily decode. Higher-dimensional embeddings contain richer information but require more sophisticated classifiers to exploit effectively. This reveals that optimal embedding size is not an intrinsic data property, but rather emerges from the interaction between encoder compression and classifier capacity. Therefore, hyperparameter selection must be guided by end-task performance across the entire pipeline, not intermediate objectives like reconstruction loss.

Part 3 – Autoencoder and Classifier Design Experiments

Objective and Design Decisions

I conducted three systematic experiments to optimise system performance: first, identifying the best autoencoder architecture for feature extraction; second, optimising the classifier to leverage those features effectively; and third, applying regularisation to improve generalisation on the small labelled dataset.

Experiment 1: Autoencoder Architecture Selection

I compared four autoencoder architectures to determine which produced the most effective features: the **Original Dense** model, a **Deeper Dense** version with additional layers, and two **Convolutional Autoencoders (CAE)** with different kernel sizes (5×5 and 3×3). All were tested using the same simple 16-unit classifier to ensure fair comparison. Table 2 summarises the results, and Figure 7 shows the classifier validation accuracy curves.

Table 2: Autoencoder architecture comparison with simple 16-unit classifier.

Architecture	AE Val. Loss	Classifier Val. Acc.
Convolutional (5×5 kernels)	0.0128	72.31%
Convolutional (3×3 kernels)	0.0129	75.95%
Original Dense	0.0188	76.99%
Deeper Dense	0.0145	78.26%

Initially, the **Deeper Dense autoencoder achieved the highest accuracy at 78.26%**, outperforming the Original Dense (76.99%) and both CAE variants (75.95% for 3×3 , 72.31% for 5×5). However, examining the reconstruction quality reveals an important insight: the convolutional autoencoders, particularly the 3×3 variant, produce significantly sharper reconstructions without the blurry artefacts present in dense models (Figure 8). Despite achieving lower reconstruction loss (0.0129 vs 0.0145), the CAE’s features resulted in lower classification accuracy when paired with the simple classifier.

This apparent contradiction suggests the CAE learns more complex and spatially-structured representations, but the simple 16-unit classifier may lack sufficient capacity to fully exploit these richer features. The 3×3 kernel outperformed the 5×5 variant, likely because smaller kernels capture finer local patterns more suitable for digit recognition. This observation motivated my next experiment to test whether a more powerful classifier could better leverage the CAE’s superior feature learning capabilities.

Experiment 2: Classifier Architecture Selection

To test whether a more capable classifier could better utilise the features from the convolutional autoencoder, I fixed the CAE as the feature extractor and explored different classifier architectures. I compared a simple baseline, a deeper architecture (two hidden layers), and a wider architecture (single large hidden layer).

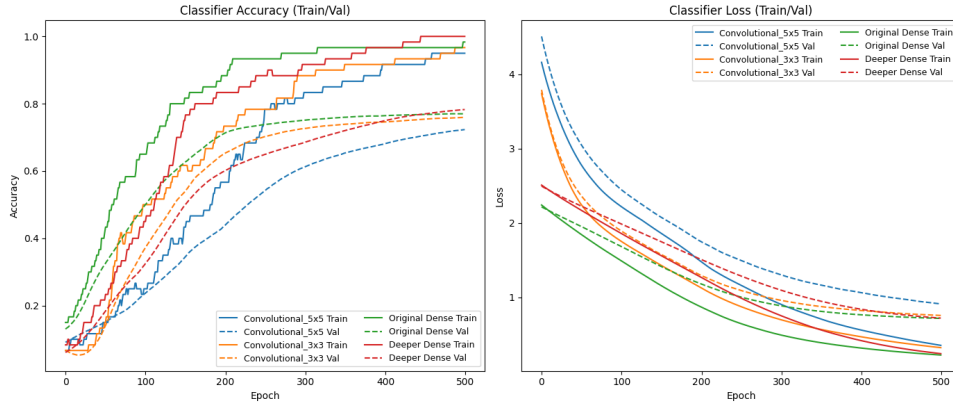


Figure 7: Classifier performance across autoencoder architectures using a simple 16-unit classifier.

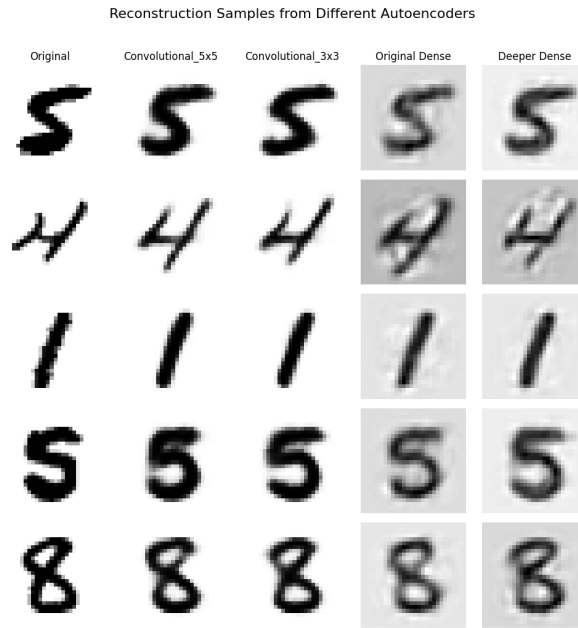


Figure 8: Reconstruction quality comparison. The convolutional autoencoder produces sharper, less blurry reconstructions.

As shown in Figure 9, the hypothesis was confirmed: the **Wide classifier with 128 units achieved 83.53% accuracy**, dramatically outperforming the simple classifier’s 75.95% with the same CAE features (Sees Table 3). This 7.58% improvement validates that the convolutional autoencoder was indeed learning higher-quality representations, but required a classifier with sufficient capacity to decode them. The wide, shallow architecture proved more effective than deeper alternatives, showing faster convergence and a higher accuracy plateau without introducing additional training difficulties.

Experiment 3: Improving Generalisation with Regularisation

Having identified the best architecture, the final experiment focused on combating overfitting. With 128 neurons in the hidden layer and only 60 training examples, the classifier has more parameters than training samples, creating substantial overfitting risk. I systematically tested L2 regularisation and Dropout, both individually and in combination.

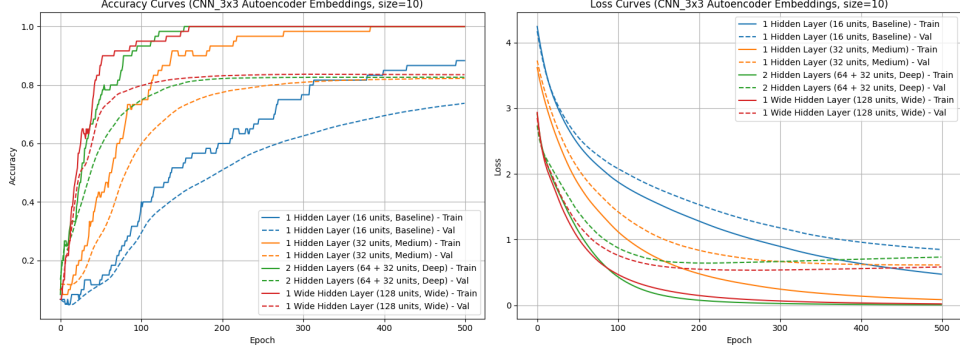


Figure 9: Classifier architecture comparison using fixed convolutional autoencoder features.

Results in Figure 10 demonstrate the effectiveness of regularisation. L2 regularisation alone improved accuracy to 84.04%, while Dropout alone achieved 82.88%. However, **combining L2 and Dropout yielded the best performance at 85.11% validation accuracy** (Sees Table 3). The combined approach exhibits more stable validation curves with reduced variance compared to either technique alone. This demonstrates that simultaneously penalising model complexity (L2) and forcing robust feature co-adaptations (Dropout) provides complementary benefits for generalisation under extreme label scarcity.

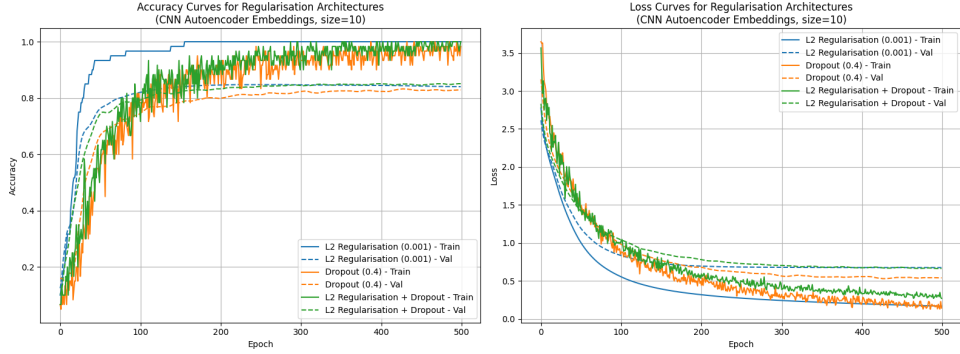


Figure 10: Regularisation technique comparison showing combined L2 and Dropout achieves optimal performance.

Summary of Results

Table 3 summarises the progressive improvement achieved through each design decision. Starting from the baseline 76.99% accuracy, the systematic experimentation yielded a final model achieving 85.11%, representing a substantial 10.55% relative improvement.

Conclusion

Through this three-stage iterative process, system performance improved significantly from 76.99% to 85.11%. The optimal design pairs a **Convolutional Autoencoder** with 3×3 kernels for spatial feature extraction with a **Wide 128-unit classifier** regularised using both L2 penalty and Dropout.

The key insight from these experiments is the importance of matching feature extractor and

Table 3: Progressive model improvements across experiments.

Model Configuration (Latent Size = 10)	Val. Acc.	Improvement
Baseline: Original Dense AE + Simple Classifier	76.99%	—
Deeper Dense AE + Simple Classifier	78.26%	+1.65%
Convolutional AE + Simple Classifier	75.95%	−1.35%
Convolutional AE + Wide Classifier	83.53%	+8.49%
Convolutional AE + Wide Classifier + L2	84.04%	+9.16%
Convolutional AE + Wide Classifier + Dropout	82.88%	+7.65%
Final: Conv. AE + Wide Classifier + L2 + Dropout	85.11%	+10.55%

classifier capacities. While the convolutional autoencoder learned superior representations compared to dense alternatives, this advantage only materialised when paired with a sufficiently powerful classifier. Furthermore, given the extreme label scarcity (60 examples), aggressive regularisation was essential to prevent overfitting and achieve robust generalisation. This systematic approach highlights that optimal performance in semi-supervised learning requires careful co-design of the entire pipeline rather than optimising components in isolation.